

A. Truyền tin

time limit per test: 1 second

memory limit per test: 256 megabytes

Đây là bài toán interactive.

Bảo đang làm tình báo, trong quá trình do thám Bảo đã thu thập được rất nhiều thông tin có giá trị và gửi về cho trụ sở chính BKCC. Tuy nhiên khi quay trở về báo cáo, Bảo phát hiện rằng tất cả dữ liệu mật báo gửi về đều đã bị lỗi và không thể sử dụng được (khiến cho Bảo bị vụt mất KPI và bị cấp trên khiển trách). Sau quá trình phân tích, Bảo phát hiện rằng do thiết bị truyền tin của mình mà tất cả thông tin mà Bảo gửi về đều có một lỗi chung như sau:

Thông tin mà Bảo gửi về có dạng là một mã nhị phân độ dài n , do thiết bị lỗi mà **chính xác một** bit trong dãy nhị phân này bị biến đổi, tức là bit 0 thành 1 và ngược lại.

Do không đủ tiền để sửa chữa thiết bị, Bảo quyết định gửi thêm thông tin để khắc phục vấn đề này. Thông tin mà Bảo gửi thêm là các *bit* được đính kèm với mật mã gốc gọi là các *xor_bit*. Mỗi *xor_bit* có thể chứa k thông tin $i_1, i_2, \dots, i_k$. Trong đoạn mật mã mới, nếu tại pos là một *xor_bit* nó sẽ có giá trị bằng với $a_{i1} \oplus a_{i2} \oplus \dots, a_{ik}$. Với a_i là giá trị của bit thứ i trong đoạn mật mã mới.

Đoạn mật mã mới sẽ là đoạn mật mã cũ nhưng được chèn vào các *xor_bit* mà bạn quy định, mà không làm mất đi thứ tự của đoạn mật mã gốc.

Do thông tin mật báo cần được bảo mật chặt chẽ, thông tin sau khi thêm các *xor_bit* mà Bảo có thể gửi không được vượt quá $n + 50$. Đầu đầu vì không biết phải thiết kế dãy *xor_bit* làm sao, Bảo nhờ tới bạn giúp Bảo chọn ra các *xor_bit*, sau đó nhận mã truyền tin của Bảo với các *xor_bit* mà bạn đã quy ước, và xác định bit nào bị lỗi trong quá trình truyền tin (hoặc là bit của mã ban đầu hoặc là *xor_bit*)

Input

Dòng đầu tiên là một số nguyên dương t ($1 \leq t \leq 100$) — Số testcases

Dòng đầu tiên của mỗi testcase gồm một số nguyên dương n ($1 \leq n \leq 5000$) — Độ dài mật mã.

Dữ liệu vào bảo đảm tổng n ở mọi testcase không vượt quá 5000.

Sau khi đọc vào input cho mỗi testacse, tiến hành quá trình interact như sau.

Interaction

Với mỗi testcase, thực hiện những điều dưới đây.

Nếu sử dụng k *xor_bit*, hãy in dưới dạng sau:

- ? c ($0 \leq c \leq 50$) — Số *xor_bit* sẽ được sử dụng.

Sau đó in ra c dòng, mỗi dòng có dạng sau:

- ? pos k $i_1, i_2, \dots, i_k$ ($0 \leq k \leq n$, $1 \leq pos \leq n + c$, $1 \leq i_j \leq n + c$, mọi i_j đều đôi một

khác nhau) - Lần lượt là vị trí của xor_bit trong mật mã bạn nhận được, số lượng thông tin mà xor_bit đó chứa và các thông tin của xor_bit đó.

Nếu $k = 0$ thì $xor_bit = 0$.

Lưu ý, các thông tin của xor_bit không thể là vị trí một xor_bit nào đó. Hay i, pos đều phải thỏa $i \neq pos$ với i là thông tin của một xor_bit và pos là vị trí của xor_bit nào đó trong mật mã mới. Đồng thời, các vị trí pos của các xor_bit cũng phải đôi một khác nhau.

Khi đã in ra đầy đủ các thông tin, hãy đọc vào $n + c$ giá trị $a_1, a_2, \dots, a_{n+c}$ ($a_i \in \{0, 1\}$) – Đoạn mật mã sau sau khi Bảo gửi cho bạn, trong đó có một bit đã bị sai thông tin so với đoạn mật mã gốc.

Nếu bạn biết được vị trí của bit bị sai, hãy in ra một dòng dưới dạng sau: $! id$ ($1 \leq id \leq n + c$).

Sau khi thực hiện một thao tác hãy nhớ in ra ký tự xuống dòng và flush output. Nếu không bạn có thể gặp `Idleness limit exceeded` hoặc các kết quả khác. Để làm điều này, hãy sử dụng:

- `fflush(stdout)` hoặc `cout.flush()` trong C++;
- `System.out.flush()` trong Java;
- `flush(output)` trong Pascal;
- `stdout.flush()` trong Python;
- xem tài liệu đối với các ngôn ngữ khác.

Khuyến khích các bạn đọc thêm tài liệu về [hướng dẫn dành cho các bài toán interactive](#).

Example

input
3 1 5 3
output
? 2 ? 1 1 3 ? 2 1 3 ! 2 ? 1 ? 1 4 2 3 4 5 6 ! 6 ? 0 ! 3

Note

Lưu ý: Đây không phải là các bước để giải bài toán này, đây là ví dụ về cách tương tác với máy chấm.

Ở ví dụ thứ nhất, nhập vào $n = 1$, lúc này đưa ra các thông tin sau:

- ? 2 — sẽ có 2 *xor_bit* được sử dụng.
- ? 1 1 3 — *xor_bit* sẽ ở vị trí 1 và ảnh hưởng đến bit thông tin ở vị trí 3.
- ? 2 1 3 — *xor_bit* sẽ ở vị trí 2 và ảnh hưởng đến bit thông tin ở vị trí 3.

Giả sử thông tin ban đầu là $[1]$, lúc này ta có thông tin mới là $[\underline{1}, \underline{1}, 1]$ (các bit được gạch chân là *xor_bit*). Tuy nhiên sau khi nhận được thông tin, bạn thu được $[\underline{1}, \underline{0}, 1]$. lúc này, bit bị sai nằm ở vị trí 2, in ra ! 2.

Ở ví dụ thứ hai, nhập vào $n = 5$, giả sử dãy bit ban đầu là $[1, 0, 0, 1, 0]$. Sau khi thêm *xor_bit*, nó trở thành $[\underline{0}, 1, 0, 0, 1, 0]$, lúc này qua truyền tin nó trở thành $[\underline{0}, 1, 0, 0, 1, 1]$. Lúc này bit bị sai nằm ở vị trí 6, in ra ! 6.

Ở ví dụ thứ ba, nhập vào $n = 3$, giả sử dãy ban đầu là $[0, 0, 1]$, bạn bằng cách thần kì nào đó không cần *xor_bit*, sau khi qua truyền tin nó trở thành $[1, 0, 1]$. Lúc này bạn in ra ! 1.